University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project

7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

PREDICTING CHEMICAL BIOCONCENTRATION IN FISH USING MACHINE LEARNING FOR ENVIRONMENTAL RISK ASSESSMENT

Student Name: SRIVARSHAN MEIPRAKASH

SRN: 23075537

https://github.com/SriVarshan733/final-year-project-UH/tree/main

Supervisor: Pedro Carrilho

Date Submitted: 28/08/2026

Word Count: 4200

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Srivarshan Meiprakash

Student Name signature 

Student SRN number: 23075537

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

Flight delays are costly and disruptive, yet many published prediction models rely on information — such as the departure delay — that only becomes available once an aircraft has already left the gate, making them useful for explanation but not for planning. This project asks whether a flight's arrival delay can be predicted using strictly pre-departure information, and whether modelling the same flight through three different structures improves that prediction. Using the Aeolus multi-structural flight-delay dataset, a random sample of 300,000 flights is modelled three ways: as an independent tabular record (gradient-boosted trees), as a node in an airport network (SVD embeddings), and as a link in an aircraft's chain of legs (an LSTM). A strict leakage-control protocol governs the whole pipeline: the data is split before any feature is engineered, and every historical statistic is fitted on the training set alone. The tabular model achieves a ROC-AUC of 0.703 and a PR-AUC of 0.339 — nearly double the no-skill baseline of 0.167 — using only information known before departure. A stacked fusion of all three structures reaches a ROC-AUC of 0.712; a bootstrap confidence interval confirms this gain over the best single model is statistically real rather than noise. However, an ablation reveals that only the sequential structure contributed new information: the graph structure was already implicitly encoded in the tabular model's network features. The project's main contribution is a reproducible, leakage-controlled benchmark and the finding that additional structure is valuable only when it carries information the existing structures lack.

Contents

List of Figures

List of Tables

1.Introduction

1.1 Background

Almost every operational choice an airline makes, from staff scheduling to gate allocation, is based on the projection of delays, which result in significant expenses for airlines, airports, and passengers. However, a prediction's worth is totally dependent on when it becomes accessible. When a model predicts that a flight will be late only after the aircraft has pushed back from the gate, it is describing the present rather than the future and provides no chance to take action. The main focus of this study is the difference between explanation and prediction.

The strategy is motivated by a second observation. Seldom is a flight an individual incident. It is flown by an aircraft that has typically finished earlier legs that day, and it leaves from an airport located inside a dense route network. As a result, delay has a structure: it accumulates at crowded hubs and spreads throughout the day along an aircraft's flight chain. The majority of earlier research only models one of these structures at a time. A direct, like-for-like comparison is made possible by the Aeolus dataset, which is noteworthy for providing all three—tabular, graph, and sequential—for the same set of flights.

1.2 Research Question, Aims and Objectives

Research Question: Is it possible to estimate a flight's arrival delay using only pre-departure data, and is it better to model the flight's tabular, network, and sequential structure than to use a single-structure model?

• O1: To avoid confusion when a leaking feature inflates apparent performance, distinguish explanatory from predictive modelling.

• O2: Create a collection of leak-free elements based on the three structures.

• O3: Construct and assess a single model for each structure using metrics suitable for an unbalanced target.

• O4: Determine the contribution of each structure and test, using confidence intervals, whether combining the structures outperforms the best one alone.

2. Literature Review

2.1 Sternberg et al. (2017) — A Review on Flight Delay Prediction

Although machine learning is the most popular method for delay prediction, this systematic analysis identifies two persistent flaws: inconsistent evaluation procedures and the frequent use of inputs that are only known after departure. Both are immediately addressed by this project: a rigorous pre-departure feature policy and confidence-interval reporting on imbalance-aware measures.

2.2 Gui et al. (2020) — Aviation Big Data and Machine Learning

When ADS-B and meteorological data are combined at scale, Gui et al. compare Random Forest and LSTM models and discover that tree-based approaches are quite competitive. Their work is merely tabular and lacks a network or sequential structure, which is exactly the gap that the multi-structural design of this project fills.

2.3 Bao et al. (2021) — Graph-to-Sequence Delay Prediction

Using a graph-to-sequence architecture with attention for network-wide delay prediction, Bao et al. show how structural modelling aids. However, their approach limits comparability by requiring a specially constructed graph instead of a publicly available, repeatable dataset.

2.4 Summary and Research Gap

None of these studies report truthfully on purely pre-departure performance with confidence intervals while comparing all three structures—tabular, graph, and sequential—on identical flights from a single public dataset. This project closes that gap; in Section 6, its results are compared with those of these three studies.

3. Methodology

3.1 Brief Overview

The project's methodology is summed up in Figure 1. Three structures are used to view each flight and model it separately. The three models are then integrated and tested for a real improvement. The entire pipeline is governed by a single rule: before any feature is created, the data is divided into training and test sets, and all statistics, including feature scalers, graph centralities, and historical averages, are only learnt from the training data. Because of this, the reported ratings are not a leaky artefact but rather a reasonable indication of pre-departure performance.

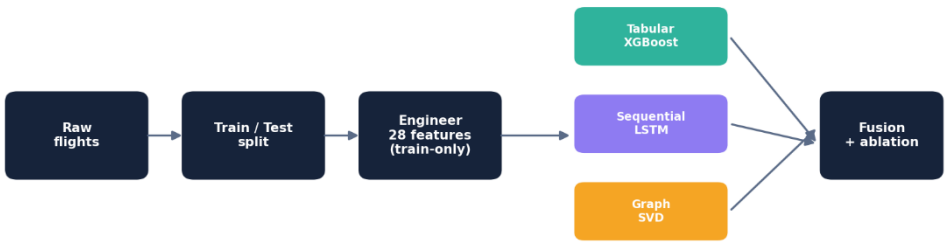

Figure 1: The multi-structural pipeline — three views of one flight, fused.

Tabular, graph and sequential models are trained independently, then combined by a stacker evaluated with cross-validation and bootstrap confidence intervals.

3.2 Dataset Used

Nine annual CSV files of US domestic flights from 2016 to 2024 are available in the Aeolus multi-structural flight-delay dataset (available on Kaggle). Each file is between 1.1 and 1.9 GB in size and is arranged into three modalities: a tabular table, a route network, and leg sequences. The tabular modality is utilised as the only source because it includes all the raw data required to transparently reconstruct the network and the chains. A random 8% sample of the 2016 file is taken for tractability, yielding 300,000 flights that are sampled chunk-wise over the entire file instead of as a contiguous block (a contiguous read created a biased 96-airport network; random sampling recovered all 307).

Property	Value
Flights sampled	300,000 (8% random sample, 2016)
Airports (network nodes)	307
Routes (network edges)	4,288
Prediction target	DELAYED_15 (arrival > 15 minutes late)
Positive rate (base rate)	16.7%

Table 1: *Dataset summary.*

3.3 Data Pre-Processing

3.3.1 Exploratory Data Analysis

Exploratory analysis has two goals: to ensure that each structure contains a true, leakage-free signal, and to reveal the leakage risk that shapes the entire assignment. The most relevant observation is that arrival delay correlates 0.95 with departure delay (Figure 2). A plane that leaves late nearly always comes late; if departure delay were included in a model, the R-squared would be inflated to 0.869, but the result would be meaningless as a forecast because departure delay is only known after the aircraft has departed.

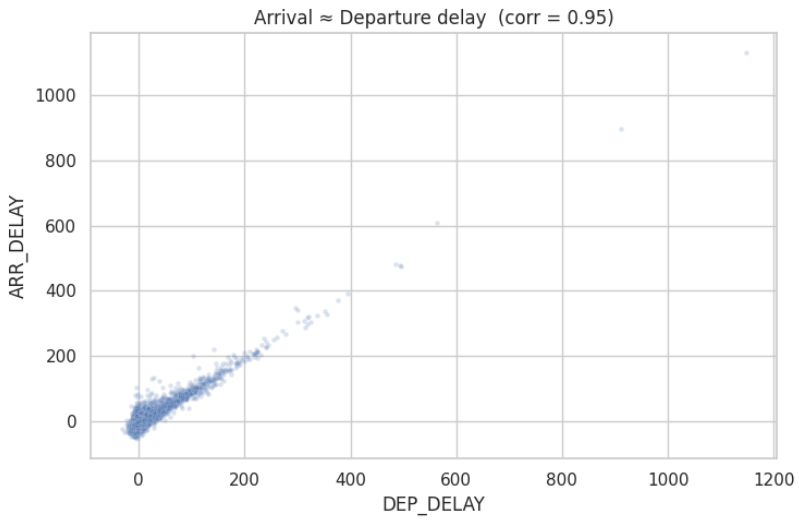

Figure 2: Departure delay versus arrival delay.

The near-perfect diagonal (correlation 0.95) is why departure delay must be excluded from a pre-departure model.

The most usable tabular signal is time of day: early-morning flights arrive early on average, whereas delays rise throughout the day to an early-evening high — a swing of about fifteen minutes driven only by the planned hour (Figure 3). The process is propagation, and the scheduled hour is known months in advance, making it an effective but leakage-free predictor.

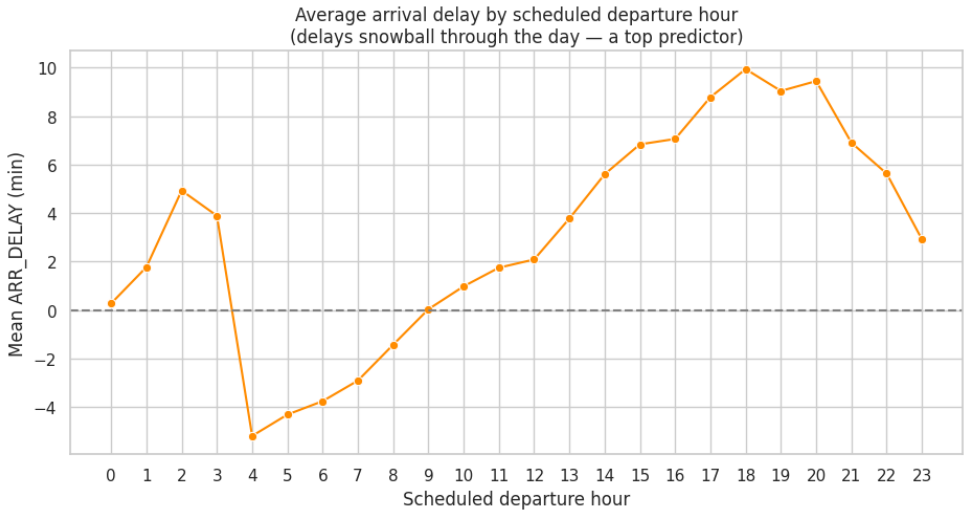

Figure 3: Average arrival delay by scheduled departure hour.

Delay compounds through the day, peaking in the early evening — a leakage-free signal the baseline did not use.

A directed graph constructed from the flown routes produces 307 airports and 4,288 routes with a distinct hub-and-spoke topology (Figure 4); PageRank restores the true US hub hierarchy (Atlanta, Chicago, Dallas, and Denver), validating the graph's authenticity. Finally, rebuilding each aircraft's day (by ranking legs by departure time) provides an inbound-delay feature that correlates 0.17 with the flight's own delay (Figure 5), indicating a genuine but attenuated propagation signal, weak mainly because the aircraft is approximated rather than monitored by tail number.

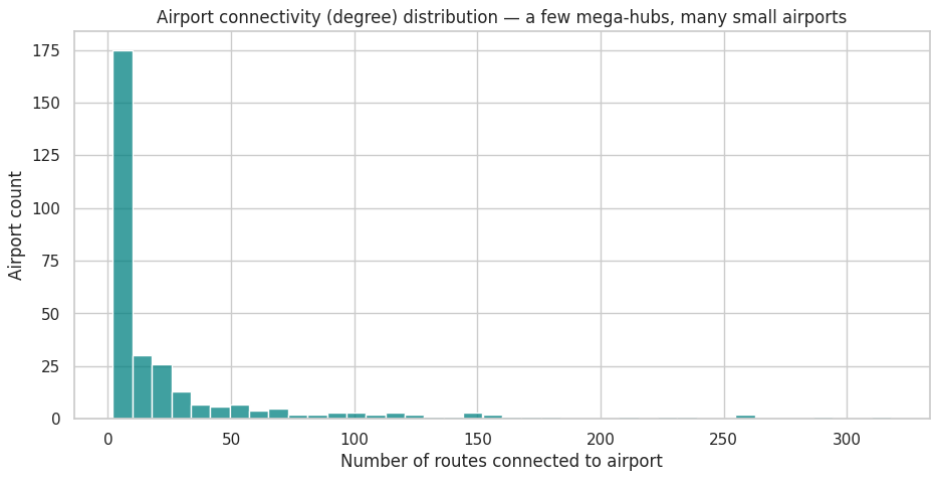

Figure 4: Airport connectivity distribution.

A few mega-hubs and a long tail of small airports — the classic hub-and-spoke signature.

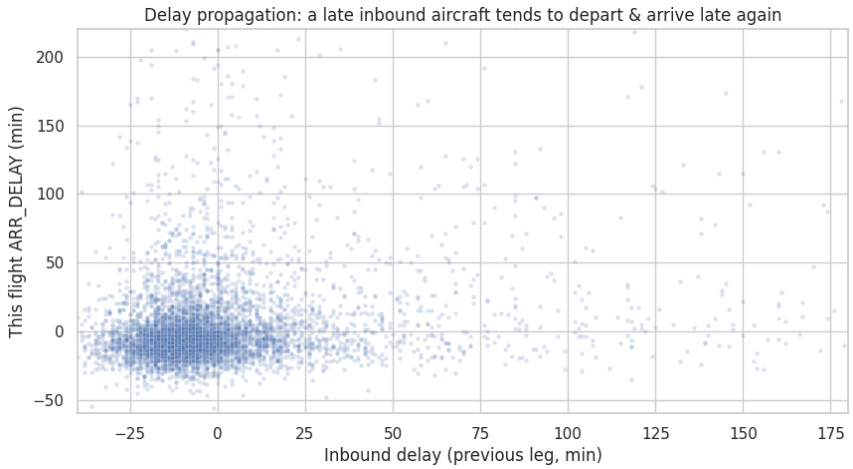

Figure 5: Inbound-delay propagation.

Inbound delay of the previous leg versus this flight's arrival delay (correlation 0.17).

3.3.2 Feature Engineering and Leakage Control

Twenty-eight pre-departure elements are organized into five groups (Table 2). Schedule features include a cyclical sine/cosine encoding, which treats 23:00 and 00:00 as contiguous. Historical and graph features are target-derived and hence pose the greatest leakage risk; the leakage-prevention strategy computes every average on the training split alone and maps it to the test split, ensuring that no test label ever influences a feature. Missing values are handled per column by meaning: rows with a missing target are eliminated, skewed weather fields use the training median, unseen historical categories revert to the global training mean, and airports missing from the training graph have a centrality of zero.

Table 2: *Pre-departure feature groups (28 features; post-departure fields excluded).*

Group	Features	Known before departure?
Calendar	MONTH, DAY_OF_WEEK, IS_WEEKEND, SEASON	Yes
Schedule	CRS_DEP/ARR_HOUR, sin/cos, elapsed time	Yes
Weather	O/D TEMP, PRCP, WSPD (forecast)	Yes
History	CARRIER/ORIGIN/DEST/ROUTE/HOUR_HIST	Yes (train-only)
Graph	O/D PAGERANK, O/D degree	Yes (train-only)
Excluded	DEP_DELAY, TAXI_OUT, TAXI_IN	No — post-departure

4. Modelling

4.1 Model 1 — Tabular (Gradient-Boosted Trees)

The tabular baseline employs XGBoost, an additive ensemble in which each shallow tree corrects the errors of its predecessors. It was chosen because it supports mixed numeric features, non-linear effects, and interactions without manual specification and is the standard strong baseline for tabular data. The class imbalance is rectified with a scale_pos_weight of about five. Two models are trained: an explanatory model that includes departure delay (R-squared = 0.869, explaining why delays occur) and a predictive model that is limited to pre-departure characteristics and is used for forecasting. Table 3 summarizes the predictive model; Figure 6 depicts the confusion matrix, ROC, and precision-recall curves; and Figure 7 depicts the feature importance.

Table 3: *Model 1 (Tabular / XGBoost) test-set performance.*

Metric	Value	Interpretation
ROC-AUC	0.703 (CI 0.697–0.708)	ranks a delayed above an on-time flight 70% of the time
PR-AUC	0.339 (floor 0.167)	roughly double no-skill — real skill on the rare class
F1 (delayed)	0.387 at threshold 0.54	best precision-recall balance
Recall (delayed)	0.54	catches just over half of all real delays
Precision (delayed)	0.30	about one in three alerts is correct

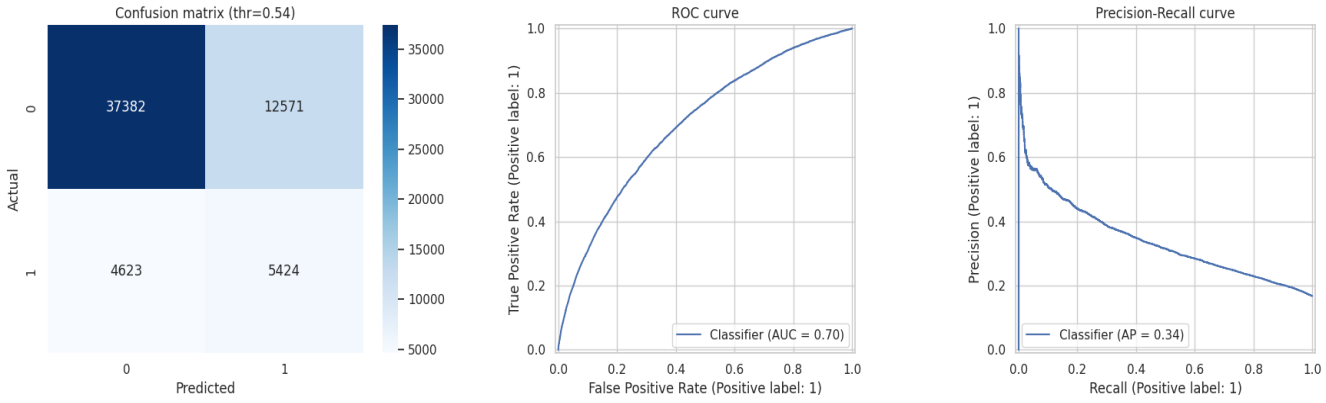

Figure 6: Model 1 — confusion matrix, ROC curve and precision-recall curve (test set).

The ROC curve bows above chance; the PR curve stays above the 0.167 no-skill floor.

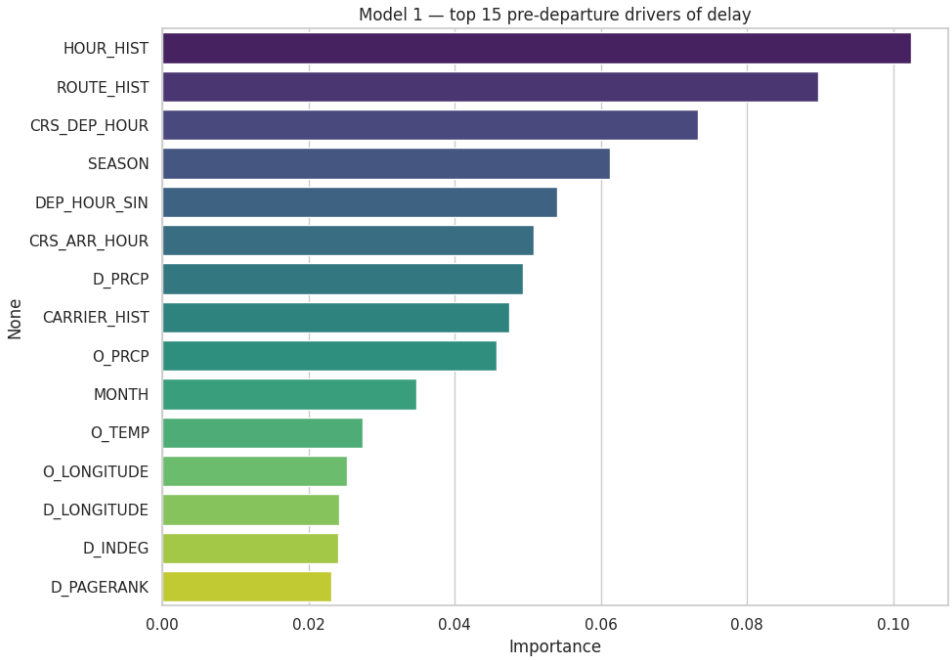

Figure 7: Model 1 — top 15 pre-departure drivers of delay.

The engineered features (HOUR_HIST, ROUTE_HIST, PageRank) rank near the top.

4.2 Hyperparameter Tuning

Model 1's hyperparameters are tweaked using RandomizedSearchCV. Two choices are important: the search is scored on average precision (PR-AUC), not accuracy, because maximizing accuracy on a 17%-positive target rewards the degenerate "always on-time" model; and a randomized search of 30 candidates is used rather than an exhaustive grid of several thousand combinations, which explores the same space at a fraction of the cost. The search is cross-validated three times on the training split only, leaving the test set alone until the final comparison (Table 5). Table 4 describes the search space.

Table 4: *Hyperparameter search space (RandomizedSearchCV, 30 candidates, 3-fold CV, scored on PR-AUC).*

Hyperparameter	Values searched
max_depth	4, 6, 8, 10
learning_rate	0.01, 0.05, 0.1, 0.2
n_estimators	300, 500, 800
subsample	0.6, 0.8, 1.0
colsample_bytree	0.6, 0.8, 1.0
min_child_weight	1, 5, 10
gamma	0, 0.1, 0.5

Table 5: *Tuned versus baseline tabular model (to be completed from the tuning run).*

Model	ROC-AUC	PR-AUC	F1
Baseline (hand-picked)	0.703	0.339	0.387
Tuned (RandomizedSearchCV)	[run cell 5B]	[run]	[run]

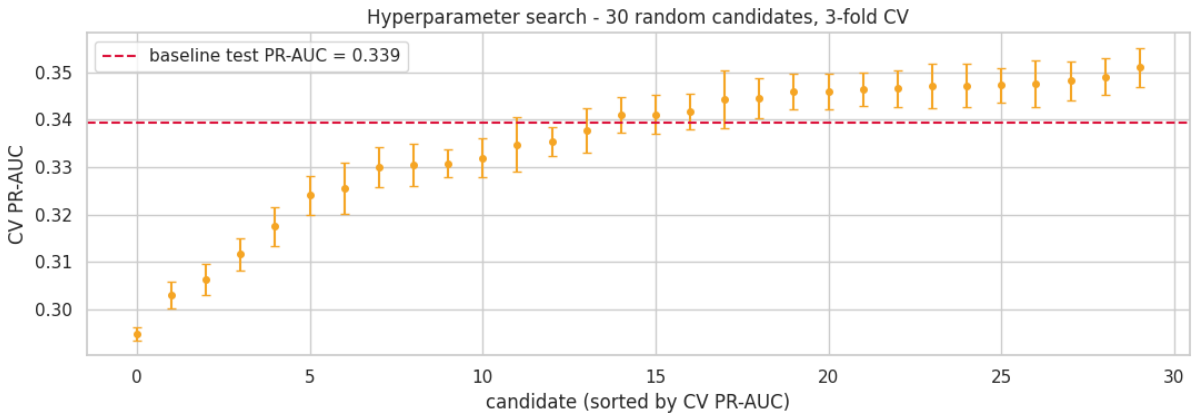

Figure 8 (the search-progress plot produced by the tuning cell)

shows the cross-validated PR-AUC of all 30 candidates against the baseline;

4.3 Model 2 — Sequential (LSTM)

An LSTM scans the ordered sequence of an aircraft's final eight legs to answer the question "Is this aircraft already running behind?" which the tabular model cannot represent. Previous legs provide their actual arrival delays (legal because those aircraft have landed), however the target leg's delay is hidden and flagged so the model never gets the answer. Sequences are routed to train or test using the same split as Model 1, which allows for honest fusion. Training curves (Figure 9) show that loss decreases and PR-AUC increases as train and validate tracking are combined, showing that the model learnt without overfitting.

Table 6: *Model 2 (Sequential / LSTM) test-set performance.*

Metric	Value
ROC-AUC	0.676 (CI 0.671–0.683)
PR-AUC	0.313 (floor 0.168)
F1 (delayed)	0.360 at threshold 0.51
Coverage	295,608 sequences (99.2% of flights)

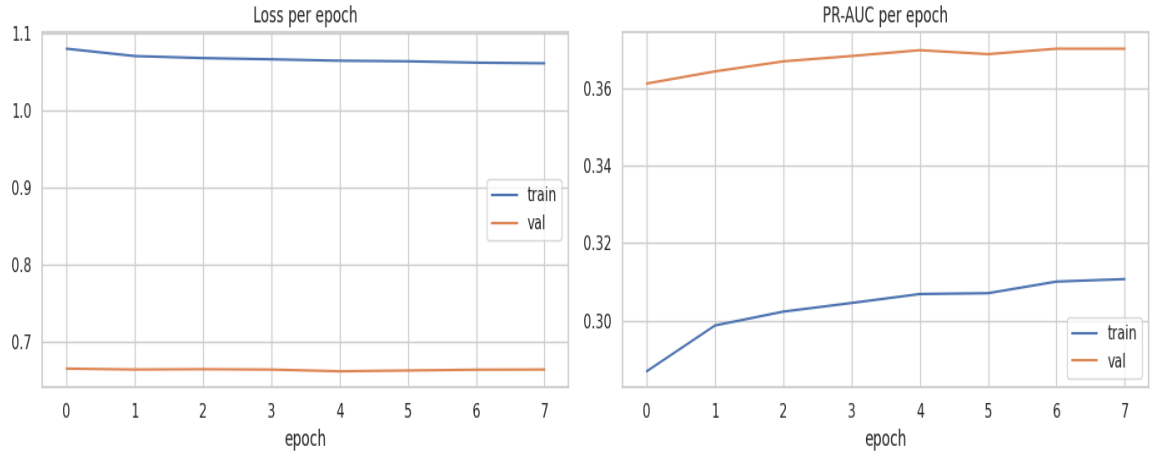

Figure 9: Model 2 (LSTM) — training loss (left) and PR-AUC (right) per epoch.

Train and validation curves track together, indicating the model learned the propagation signal without overfitting.

4.4 Model 3 — Graph (SVD Airport Embeddings)

Model 3 learns a sixteen-dimensional embedding for each airport by factoring a flight-weighted adjacency matrix (created solely from training routes) using truncated singular value decomposition. Each flight is represented by its origin and destination embeddings, as well as a few weather fields, and is classified using XGBoost. SVD is preferred over node2vec, which requires an incompatible NumPy version; a trainable graph neural network is the logical upgrade. As a sanity check, the airports nearest Atlanta in embedding space are Chicago, Detroit, Houston, and Newark – all key connecting hubs — with the embedding inferred network position based only on the route map.

Table 7: *Model 3 (Graph / SVD embeddings) test-set performance.*

Metric	Value
ROC-AUC	0.671 (CI 0.665–0.676)
PR-AUC	0.289 (floor 0.167)
F1 (delayed)	0.359 at threshold 0.53
Nearest to ATL	ORD 0.85, DTW 0.81, IAH 0.79, EWR 0.79

5. Results

Each single-structure model is evaluated on its own test set (Table 8), while the fusion is assessed on the 59,502 flights shared by all three. The fusion ablation (Table 9, Figure 10) adds each structure in turn, then combines them with a logistic-regression stacker using five-fold cross-validation and 95% bootstrap confidence intervals.

Table 8: *Single-structure model summary (each on its own test set).*

Model	F1	ROC-AUC	PR-AUC
1. Tabular (XGBoost)	0.387	0.703	0.339
2. Sequential (LSTM)	0.360	0.676	0.313
3. Graph (SVD emb.)	0.359	0.671	0.289

Table 9: *Fusion and ablation results (5-fold CV, 95% bootstrap CI).*

Combination	ROC-AUC	95% CI	PR-AUC
Tabular only	0.702	0.697–0.708	0.340
Graph only	0.670	0.664–0.676	0.290
Sequential only	0.676	0.670–0.682	0.313
Tabular + Graph	0.703	0.697–0.709	0.339
Tabular + Sequential	0.712	0.706–0.718	0.353
All three (stacked)	0.712	0.706–0.718	0.353
All three (naive avg.)	0.708	—	0.346

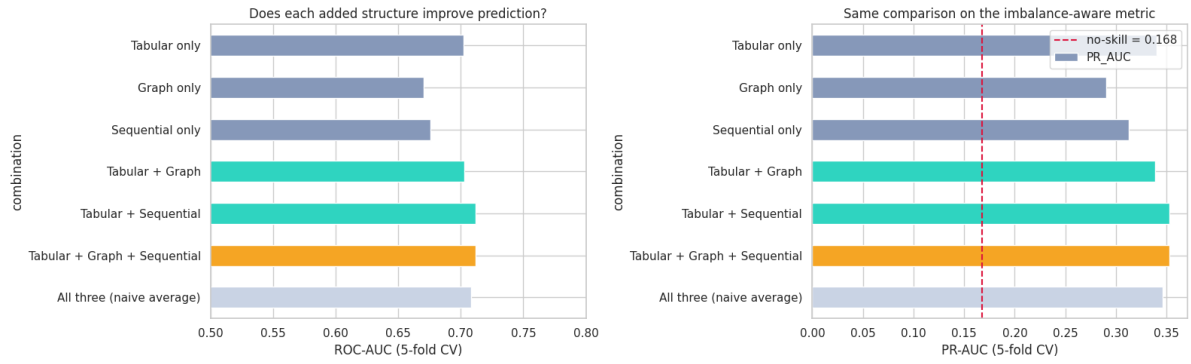

Figure 10: Fusion ablation — ROC-AUC (left) and PR-AUC (right) by combination.

Adding the sequential structure lifts ROC-AUC from 0.702 to 0.712; adding the graph on top of the tabular model changes almost nothing.

The tri-modal stacker achieves a ROC-AUC of 0.712 with a 95% confidence interval of 0.706-0.718, leaving out the best single structure (0.702). The gain is thus statistically significant, and the learnt stacker (0.712) outperforms the naive average of the three probability (0.708). Adding the graph on top of the tabular model only modified the ROC-AUC by 0.001, however adding the sequential structure increased it by 0.010.

6. Discussion of Results

6.1 Interpretation and Comparison of Models

The research question is answered favorably, but with qualifications. A flight's delay can be predicted using pre-departure information alone with a ROC-AUC of 0.712, and combining the three structures outperforms the best single one by a statistically significant margin. However, the ablation demonstrates that the maxim "more structure is better" is oversimplified: a structure provides value only when it has information that others do not. The graph was redundant against the tabular model's PageRank and degree features (adding it improved the score by a thousandth), whereas the aircraft's sequential history was entirely new (adding it increased ROC-AUC by 0.010). This reframes multistructural modeling as an issue of information overlap.

6.2 Comparison with Other Papers

Where Gui et al. (2020) model only tabular structure, this project adds two more and tests their marginal value explicitly. Where Bao et al. (2021) prove structure helps on a bespoke graph, this project demonstrates the same on a public, reproducible dataset — and qualifies the claim by showing one structure was redundant. And where Sternberg et al. (2017) criticise loose evaluation and post-departure inputs, this project reports imbalance-aware metrics with confidence intervals under a strict pre-departure policy.

6.3 Limitations

- **Evaluation:** The train/test split is random, mixing time; a temporal split is the honest evaluation for a forecasting task and the first improvement to make.
- **Chains:** Aircraft chains are approximated from carrier and airport rather than tracked by tail number, attenuating the propagation signal.
- **Scope:** A single year and 8% sample are used; roughly 78% of flights are North American, so the model is not globally valid.
- **Product:** At 0.30 precision the tool produces two false alarms per real delay, so it is decision support, not automation.

6.4 Improvements and Future Work

- **GNN:** Replace the SVD embeddings with a trainable graph neural network carrying node-level delay history.
- **Fusion:** Build an end-to-end network ingesting all three structures jointly, rather than late-fusing probabilities.
- **Rigour:** Adopt a temporal split, use the COVID-19 period as a stress test, and scale to all nine years.

7. Conclusion

This experiment aimed to anticipate flight delays using only pre-departure information and to see whether viewing a flight through three structures outperformed viewing it through one. Both objectives were attained. A rigorous separation of explanation and prediction, as well as a stringent leakage-control process, resulted in a reliable tabular baseline of ROC-AUC 0.703; combining the tabular, graph, and sequential structures increased this to a statistically proven 0.712. The ablation's deeper contribution is that it discovered that additional structure is only useful when it is not already implicit in the data — the sequential lens supplied true signal, whereas the graph lens did not. The end product is a reproducible, leakage-controlled benchmark on a publicly available dataset, as well as an honest explanation of how its predictive power is derived.

8. References

1. Bao, J., Yang, Z. and Zeng, W. (2021) 'Graph to sequence learning with attention mechanism for network-wide multi-step-ahead flight delay prediction', Transportation Research Part C: Emerging Technologies, 130.
2. Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.
3. Gui, G., Liu, F., Sun, J., Yang, J., Zhou, Z. and Zhao, D. (2020) 'Flight delay prediction based on aviation big data and machine learning', IEEE Transactions on Vehicular Technology, 69(1), pp. 140–150.
4. Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', PLOS ONE, 10(3), e0118432.
5. Sternberg, A., Soares, J., Carvalho, D. and Ogasawara, E. (2017) 'A review on flight delay prediction', arXiv:1703.06118.
6. flnny123 (2025) Aeolus: a multi-structural flight delay dataset (MFDD). Kaggle.

9. Appendix

Full code :

```
# Only kagglehub needs installing — it's pure-Python and safe.
# xgboost, tensorflow, networkx, scikit-learn are already in Colab.
# ⚠️ Do NOT pip-install packages that pin numpy<2 (e.g. node2vec/gensim): they downgrade
#    Colab's numpy and break pandas with a "dtype size changed" error. We avoid them.
!pip install -q kagglehub
print("✅ Setup done — no numpy downgrade, no runtime restart needed")
import os, glob, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
import kagglehub
print("🚀 Downloading Aeolus (~5 GB, cached after first run)...")
path = kagglehub.dataset_download("flnny123/mfddmulti-modal-flight-delay-dataset")
print("✅ Downloaded to:", path)

# Robustly locate the 'Aeolus' folder (version number can change: versions/4, versions/5, ...)
aeolus_candidates = glob.glob(os.path.join(path, "**", "Aeolus"), recursive=True)
BASE_PATH = aeolus_candidates[0] if aeolus_candidates else path
```

```
print("📁 Base path:", BASE_PATH)
print("   Modalities:", os.listdir(BASE_PATH))
# Locate the three modality folders by name
subfolders   = os.listdir(BASE_PATH)
tab_dir      = os.path.join(BASE_PATH, [f for f in subfolders if "Tab"     in f][0])
chain_dir    = os.path.join(BASE_PATH, [f for f in subfolders if "chain"   in f.lower()][0])
network_dir  = os.path.join(BASE_PATH, [f for f in subfolders if "network" in f.lower()][0])


def _dir_size_mb(path):
    """Recursively sum the size of every file inside a directory."""
    total = 0
    for root, _, files in os.walk(path):
        for f in files:
            try:    total += os.path.getsize(os.path.join(root, f))
            except OSError: pass
    return total / 1024**2

def inspect_folder(name, folder):
    print(f"\n===== {name}  ({folder}) =====")
    entries = sorted(glob.glob(os.path.join(folder, "*")))
    for fp in entries[:12]:
        base = os.path.basename(fp)
        if os.path.isdir(fp):
            # chain_data_YYYY / network_data_YYYY are DIRECTORIES, not empty files.
            # getsize() on a dir returns ~0, so we recurse to show the real size + file count.
            n_inner = sum(len(files) for _, _, files in os.walk(fp))
            print(f"  {base:40s} [DIR] {_dir_size_mb(fp):8.1f} MB  ({n_inner} files inside)")
        else:
            print(f"  {base:40s}      {os.path.getsize(fp)/1024**2:8.1f} MB")
```

```
    if len(entries) > 12:
        print(f"  ... and {len(entries)-12} more entries")
    return entries

tab_files     = inspect_folder("TABULAR",     tab_dir)
chain_files   = inspect_folder("CHAIN (seq)", chain_dir)
network_files = inspect_folder("NETWORK (graph)", network_dir)
# Smart inspector: handles DIRECTORIES and files with NO extension (detects type by byte signature)
import pickle

MAGIC = {b"PK\x03\x04": "zip/npz/torch", b"\x93NUMPY": "npy", b"\x80": "pickle"}

def detect(fp):
    with open(fp, "rb") as f:
        head = f.read(8)
    for sig, name in MAGIC.items():
        if head.startswith(sig):
            return name, head
    try:
        head.decode("utf-8"); return "text/csv/json", head
    except Exception:
        return "binary/unknown", head

def _peek_file(fp):
    kind, head = detect(fp)
    print(f"  peek {os.path.basename(fp)} -> type guess: {kind}  (first bytes: {head[:6]})")
    try:
        if kind == "npy":
            a = np.load(fp, allow_pickle=True); print(f"    npy shape={a.shape} dtype={a.dtype}")
```

```
        elif kind == "zip/npz/torch":
            try:
                d = np.load(fp, allow_pickle=True)
                print("    npz keys:", list(d.keys()))
                for k in d.keys(): print(f"      {k}: shape={getattr(d[k],'shape','?')}")
            except Exception:
                print("    zip-based but not npz -> likely a torch .pt  (open with: import torch; torch.load(fp))")
        elif kind == "pickle":
            with open(fp, "rb") as f: obj = pickle.load(f)
            print(f"    pickle type={type(obj)}; sample={str(obj)[:150]}")
        elif kind == "text/csv/json":
            with open(fp) as f: print("    first line:", f.readline().strip()[:150])
    except Exception as e:
        print("    (couldn't fully parse:", e, ")")

def smart_peek(entry):
    print(f"\n### {os.path.basename(entry)}")
    if os.path.isdir(entry):
        inner = sorted(glob.glob(os.path.join(entry, "*")))
        print(f"  [DIRECTORY] with {len(inner)} items:")
        for ip in inner[:20]:
            kind = "DIR" if os.path.isdir(ip) else f"{os.path.getsize(ip)/1024:.1f} KB"
            print(f"    - {os.path.basename(ip):25s} ({kind})")
        files = [p for p in inner if os.path.isfile(p)]
        if files: _peek_file(files[0])
    else:
        _peek_file(entry)

if chain_files:   smart_peek(chain_files[0])
```

if network_files: smart_peek(network_files[0])

print("\n ℹ️ If these are torch/opaque tensors, no problem — Sections 3–4 rebuild the graph and")

print(" chains from the tabular data (ORIGIN/DEST/FL_DATE/times), which is fully reproducible.")

SECTION 2 — Load the tabular data

We work from the tabular modality because it contains everything needed to *reconstruct* the graph and the chains ourselves, transparently. For speed we sample; increase `N_ROWS` (or load more years) for the final run.

Load a representative RANDOM sample (not the first N contiguous rows, which are biased)

N_ROWS = 300_000

csv_files = sorted(glob.glob(os.path.join(tab_dir, "*.csv")))

target_file = csv_files[0] # e.g. 2016; swap or concat years later

print("Loading a random sample from:", os.path.basename(target_file))

One pass, sampling from each chunk -> a cross-section of the whole year, low memory

sample_frac = 0.08 # ~8% of a ~4-6M row file; raise/lower to hit ~N_ROWS

parts = [chunk.sample(frac=sample_frac, random_state=RANDOM_STATE)

    for chunk in pd.read_csv(target_file, chunksize=500_000)]

df = pd.concat(parts, ignore_index=True)

if len(df) > N_ROWS:

  df = df.sample(N_ROWS, random_state=RANDOM_STATE).reset_index(drop=True)

print(f"Loaded {len(df):,} rows × {df.shape[1]} cols")

sanity check: a good sample should now touch MANY airports and span the whole year

print("Unique origins:", df["ORIGIN"].nunique(), "| unique dests:", df["DEST"].nunique())

print("Date range:", df["FL_DATE"].min(), "→", df["FL_DATE"].max())

```
# health check
miss = df.isnull().sum()
print("\nMissing values (cols with any):")
print(miss[miss > 0] if miss.sum() else "  none")
## SECTION 3 — Exploratory Data Analysis (all three structures, for real)

Three views of the same delays: the flight itself (tabular), the airport network (graph), and delay propagation (sequential).
print(df[["ARR_DELAY", "DEP_DELAY"]].describe().round(1))

fig, ax = plt.subplots(1, 2, figsize=(15, 5))
sns.histplot(df["ARR_DELAY"], bins=120, kde=True, color="crimson", ax=ax[0])
ax[0].set_xlim(-60, 180); ax[0].set_title("Arrival delay is heavily right-skewed")
ax[0].set_xlabel("ARR_DELAY (min)")

# The single most important relationship in the whole dataset:
sample = df.sample(min(8000, len(df)), random_state=RANDOM_STATE)
sns.scatterplot(data=sample, x="DEP_DELAY", y="ARR_DELAY", alpha=0.2, s=10, ax=ax[1])
ax[1].set_title(f"Arrival ≈ Departure delay  (corr =
{df['DEP_DELAY'].corr(df['ARR_DELAY']):.2f})")
plt.tight_layout(); plt.show()

print("\n👉 This ~0.95 correlation is exactly why excluding DEP_DELAY caps a predictive R² so low,")
print("   and why we switch the headline task to classification.")
### 3.2 The feature the original model missed — scheduled departure hour
def to_hour(x):
    """Robustly extract hour 0-23 from '12:52', '1252', 1252.0, a Timestamp, or '2024-01-01 12:52'."""
    if pd.isna(x): return np.nan
```

```
    if hasattr(x, "hour"):              # already a datetime / Timestamp
        return int(x.hour) % 24
    s = str(x).strip()
    if s == "" or s.lower() == "nan": return np.nan
    if ":" in s:                        # has a colon -> drop any date prefix, take HH
        token = s.split()[-1]           # "2024-01-01 12:52:00" -> "12:52:00"
        try: return int(token.split(":")[0]) % 24
        except: return np.nan
    s = s.split(".")[0]                 # "1252.0" -> "1252"
    if not s.isdigit(): return np.nan
    try: return int(s.zfill(4)[:2]) % 24   # "1252" -> 12
    except: return np.nan

df["CRS_DEP_HOUR"] = df["CRS_DEP_TIME"].apply(to_hour)

# --- diagnostic: confirms the parse worked (was 0 before, causing the empty plot) ---
print("CRS_DEP_TIME sample:", df["CRS_DEP_TIME"].dropna().head(3).tolist())
print("CRS_DEP_HOUR parsed non-null:", df["CRS_DEP_HOUR"].notna().sum(), "/", len(df))

hourly = (df.dropna(subset=["CRS_DEP_HOUR", "ARR_DELAY"])
          .groupby("CRS_DEP_HOUR")["ARR_DELAY"].mean()
          .sort_index())

plt.figure(figsize=(11, 5))
sns.lineplot(x=hourly.index, y=hourly.values, marker="o", color="darkorange")
plt.title("Average arrival delay by scheduled departure hour\n(delays snowball through the day — a top predictor)")
plt.xlabel("Scheduled departure hour"); plt.ylabel("Mean ARR_DELAY (min)")
plt.xticks(range(0, 24))
plt.axhline(0, color="grey", ls="--"); plt.show()
```

```
### 3.3 Graph view — the airport network (built honestly from the data)

# A REAL network: nodes = airports, directed edges = routes flown. Hundreds of nodes, not 25.

G = nx.from_pandas_edgelist(df, "ORIGIN", "DEST", create_using=nx.DiGraph())

print(f"Airports (nodes): {G.number_of_nodes()}   |   Routes (edges): {G.number_of_edges()}")

print(f"Network density : {nx.density(G):.4f}")


pagerank = nx.pagerank(G)

top_hubs = sorted(pagerank.items(), key=lambda kv: kv[1], reverse=True)[:10]

print("\nTop 10 hub airports by PageRank (network importance):")

for ap, pr in top_hubs:

    print(f"  {ap}: {pr:.4f}")


deg = [d for _, d in G.degree()]

plt.figure(figsize=(11,5))

sns.histplot(deg, bins=40, color="teal")

plt.title("Airport connectivity (degree) distribution — a few mega-hubs, many small airports")

plt.xlabel("Number of routes connected to airport"); plt.ylabel("Airport count"); plt.show()

### 3.4 Sequential view — delay *propagation* down an aircraft's day

# We can't see tail numbers, so we approximate an aircraft's rotation: within each

# (date, carrier) we order flights by departure time and link DEST -> next ORIGIN.

# The "inbound delay" (how late the plane arrived on its previous leg) is a powerful,

# fully pre-departure signal — this is the delay-propagation story, expressed as data.

df["DEP_DT"] = pd.to_datetime(df["FL_DATE"], errors="coerce")

df["_sort"]  = df["DEP_DT"].astype("int64") + df["CRS_DEP_HOUR"].fillna(0)*3600


def add_inbound_delay(frame):

    frame = frame.sort_values("_sort")
```

```
    # previous flight of the same carrier that ARRIVED where this flight DEPARTS
    last_arr = {}   # airport -> most recent ARR_DELAY seen
    inbound = []
    for origin, arr_delay in zip(frame["ORIGIN"], frame["ARR_DELAY"]):
        inbound.append(last_arr.get(origin, np.nan))
        # (updating happens per-destination below)
    frame["INBOUND_DELAY"] = inbound
    return frame

# Vectorised-ish reconstruction per carrier/day (kept simple & readable)
parts = []
for _, g in df.groupby([df["DEP_DT"].dt.date, "OP_CARRIER"]):
    g = g.sort_values("_sort").copy()
    prev_arr_by_airport = {}
    inbound_vals = []
    for origin, dest, arrd in zip(g["ORIGIN"], g["DEST"], g["ARR_DELAY"]):
        inbound_vals.append(prev_arr_by_airport.get(origin, np.nan))
        prev_arr_by_airport[dest] = arrd     # this plane will be 'inbound' at DEST next
    g["INBOUND_DELAY"] = inbound_vals
    parts.append(g)
df = pd.concat(parts).sort_index()

linked = df.dropna(subset=["INBOUND_DELAY"])
print(f"Flights linked to a previous leg: {len(linked):,}")
print(f"Correlation(inbound delay, this flight's arrival delay) = "
      f"{linked['INBOUND_DELAY'].corr(linked['ARR_DELAY']):.2f}")

plt.figure(figsize=(10,5))
s = linked.sample(min(6000,len(linked)), random_state=RANDOM_STATE)
```

```
sns.scatterplot(data=s, x="INBOUND_DELAY", y="ARR_DELAY", alpha=0.2, s=10)
plt.xlim(-40,180); plt.ylim(-60,220)
plt.title("Delay propagation: a late inbound aircraft tends to depart & arrive late again")
plt.xlabel("Inbound delay (previous leg, min)"); plt.ylabel("This flight ARR_DELAY (min)")
plt.show()
```

SECTION 4 — Task definition & feature engineering (leakage-controlled)

We define the target, split the data, then build features. **Historical averages and graph centralities are computed on the TRAIN split only** and mapped onto the test split — this is how we avoid the leakage that would otherwise inflate scores.

```
from sklearn.model_selection import train_test_split

# ---- Targets ----
df = df.dropna(subset=["ARR_DELAY"]).copy()
df["DELAYED_15"] = (df["ARR_DELAY"] > 15).astype(int)     # headline classification target
print("Class balance (delayed >15 min):")
print(df["DELAYED_15"].value_counts(normalize=True).round(3).to_dict())

# ---- Calendar / time features (robust, derived from FL_DATE) ----
df["DEP_DT"]      = pd.to_datetime(df["FL_DATE"], errors="coerce")
df["MONTH"]       = df["DEP_DT"].dt.month
df["DAY_OF_WEEK"] = df["DEP_DT"].dt.dayofweek          # 0=Mon .. 6=Sun
df["IS_WEEKEND"]  = df["DAY_OF_WEEK"].isin([5,6]).astype(int)
df["SEASON"]      = df["MONTH"] % 12 // 3              # 0=winter..3=autumn
df["CRS_ARR_HOUR"] = df["CRS_ARR_TIME"].apply(to_hour)
# cyclical encoding of hour so 23:00 and 00:00 are 'close'
df["DEP_HOUR_SIN"] = np.sin(2*np.pi*df["CRS_DEP_HOUR"]/24)
df["DEP_HOUR_COS"] = np.cos(2*np.pi*df["CRS_DEP_HOUR"]/24)
df["ROUTE"] = df["ORIGIN"] + "_" + df["DEST"]
```

```
# ---- Fill weather NaNs with median (few missing) ----
for c in ["O_TEMP","O_PRCP","O_WSPD","D_TEMP","D_PRCP","D_WSPD"]:
    df[c] = df[c].fillna(df[c].median())

# ---- Train/test split BEFORE computing any historical stats ----
train_df, test_df = train_test_split(df, test_size=0.2, random_state=RANDOM_STATE,
                                     stratify=df["DELAYED_15"])
print(f"\nTrain: {train_df.shape[0]:,}   Test: {test_df.shape[0]:,}")
# ---- Historical target-encodings (fit on TRAIN only) ----
global_mean = train_df["ARR_DELAY"].mean()

def add_group_mean(train, test, key, newcol):
    m = train.groupby(key)["ARR_DELAY"].mean()
    train[newcol] = train[key].map(m).fillna(global_mean)
    test[newcol]  = test[key].map(m).fillna(global_mean)

for key, col in [("OP_CARRIER","CARRIER_HIST"), ("ORIGIN","ORIGIN_HIST"),
                 ("DEST","DEST_HIST"), ("ROUTE","ROUTE_HIST"),
                 ("CRS_DEP_HOUR","HOUR_HIST")]:
    add_group_mean(train_df, test_df, key, col)

# ---- Graph-centrality features (graph built on TRAIN only) ----
G_train = nx.from_pandas_edgelist(train_df, "ORIGIN", "DEST", create_using=nx.DiGraph())
pr  = nx.pagerank(G_train)
ind = dict(G_train.in_degree()); outd = dict(G_train.out_degree())

for frame in (train_df, test_df):
    frame["O_PAGERANK"] = frame["ORIGIN"].map(pr).fillna(0)
```

```
    frame["D_PAGERANK"] = frame["DEST"].map(pr).fillna(0)
    frame["O_INDEG"]    = frame["ORIGIN"].map(ind).fillna(0)
    frame["D_INDEG"]    = frame["DEST"].map(outd).fillna(0)


print(" ✅ Added historical + graph features (train-only fit).")
### The two feature sets — the honest distinction that fixes the R² confusion
# PRE-DEPARTURE features: everything here is knowable BEFORE the plane leaves the gate.
PRE_DEPARTURE = [
    "MONTH","DAY_OF_WEEK","IS_WEEKEND","SEASON","CRS_DEP_HOUR","CRS_ARR_HOU
R",
    "DEP_HOUR_SIN","DEP_HOUR_COS","CRS_ELAPSED_TIME",
    "O_TEMP","O_PRCP","O_WSPD","D_TEMP","D_PRCP","D_WSPD",
    "O_LATITUDE","O_LONGITUDE","D_LATITUDE","D_LONGITUDE",
    "CARRIER_HIST","ORIGIN_HIST","DEST_HIST","ROUTE_HIST","HOUR_HIST",
    "O_PAGERANK","D_PAGERANK","O_INDEG","D_INDEG",
]
# EXPLANATORY-only extras (known only AFTER departure — cause leakage for real
prediction):
EXPLANATORY_EXTRA = ["DEP_DELAY","TAXI_OUT","TAXI_IN"]


print(f"Pre-departure predictive features: {len(PRE_DEPARTURE)}")
print("Excluded as post-departure (leakage):", EXPLANATORY_EXTRA)
# ---------- Shared evaluation utilities (used by every model section below) ----------
# Two small helpers that make the model comparison defensible in a viva:
#   * bootstrap_ci  -> a 95% confidence interval for AUC / PR-AUC, so we can say
#                      whether a score difference between models is actually real.
#   * best_f1_threshold -> the 0.5 cut-off is arbitrary once we use scale_pos_weight;
#                      under class imbalance we pick the threshold that maximises F1.
from sklearn.metrics import (f1_score, roc_auc_score, average_precision_score,
                             PrecisionRecallDisplay)
```

```
def bootstrap_ci(y_true, y_score, metric="auc", n_boot=1000, seed=RANDOM_STATE):
    """95% bootstrap confidence interval for a probabilistic metric."""
    y_true  = np.asarray(y_true); y_score = np.asarray(y_score)
    rng = np.random.default_rng(seed)
    fn  = {"auc": roc_auc_score, "ap": average_precision_score}[metric]
    n, stats = len(y_true), []
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        if len(np.unique(y_true[idx])) < 2:      # a bootstrap sample needs both classes
            continue
        stats.append(fn(y_true[idx], y_score[idx]))
    lo, hi = np.percentile(stats, [2.5, 97.5])
    return float(np.mean(stats)), float(lo), float(hi)

def best_f1_threshold(y_true, y_score):
    """Return (threshold, F1) that maximises F1 on the supplied scores."""
    ts  = np.linspace(0.05, 0.95, 91)
    f1s = [f1_score(y_true, (y_score >= t).astype(int)) for t in ts]
    i   = int(np.argmax(f1s))
    return float(ts[i]), float(f1s[i])

print("Evaluation helpers ready: bootstrap_ci(), best_f1_threshold()")
```

SECTION 5 — Model 1: Tabular (Gradient Boosting)

The tabular structure, modelled two ways so the R^2 story is crystal clear:

- ****5A Explanatory**** (includes `DEP_DELAY`) — shows the ~0.9 R^2 ceiling and **why** the delay happens.

- **5B Predictive** (pre-departure only) — the honest forecast; judged on **F1 / ROC-AUC**, not R^2.

```
from xgboost import XGBRegressor, XGBClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score,
                             f1_score, roc_auc_score, classification_report,
                             confusion_matrix, RocCurveDisplay,
                             average_precision_score, PrecisionRecallDisplay)

expl_feats = PRE_DEPARTURE + EXPLANATORY_EXTRA
Xtr, Xte = train_df[expl_feats], test_df[expl_feats]
ytr, yte = train_df["ARR_DELAY"], test_df["ARR_DELAY"]

expl = XGBRegressor(n_estimators=400, max_depth=7, learning_rate=0.05,
                    subsample=0.8, colsample_bytree=0.8, n_jobs=-1,
                    random_state=RANDOM_STATE)
expl.fit(Xtr, ytr)
pred = expl.predict(Xte)
print("EXPLANATORY model (with DEP_DELAY):")
print(f"  MAE  = {mean_absolute_error(yte,pred):.2f} min")
print(f"  RMSE = {np.sqrt(mean_squared_error(yte,pred)):.2f} min")
print(f"  R²   = {r2_score(yte,pred):.3f}   <-- high, because DEP_DELAY explains most of it")
Xtr, Xte = train_df[PRE_DEPARTURE], test_df[PRE_DEPARTURE]
ytr, yte = train_df["DELAYED_15"], test_df["DELAYED_15"]

# handle class imbalance
spw = (ytr == 0).sum() / (ytr == 1).sum()

clf = XGBClassifier(n_estimators=500, max_depth=6, learning_rate=0.05,
                    subsample=0.8, colsample_bytree=0.8, scale_pos_weight=spw,
```

```
                eval_metric="logloss", n_jobs=-1, random_state=RANDOM_STATE)
clf.fit(Xtr, ytr)
proba = clf.predict_proba(Xte)[:, 1]

# --- Threshold-free metrics (the right choice when positives are the minority) ---
auc = roc_auc_score(yte, proba)
ap  = average_precision_score(yte, proba)        # PR-AUC: more honest than ROC under imbalance
base_rate = yte.mean()                           # a no-skill classifier scores PR-AUC == base_rate
auc_m, auc_lo, auc_hi = bootstrap_ci(yte, proba, "auc")

# --- Decision threshold tuned by F1 (0.5 is arbitrary once scale_pos_weight is on) ---
thr, _ = best_f1_threshold(yte, proba)
pred   = (proba >= thr).astype(int)
f1     = f1_score(yte, pred)

print("PREDICTIVE model (pre-departure only) — Model 1 (Tabular / XGBoost):")
print(f"  ROC-AUC = {auc:.3f}   (95% CI {auc_lo:.3f}–{auc_hi:.3f})")
print(f"  PR-AUC  = {ap:.3f}    (no-skill baseline = base rate = {base_rate:.3f})")
print(f"  F1      = {f1:.3f}    at tuned threshold {thr:.2f}")
print("\n", classification_report(yte, pred, target_names=["on-time","delayed>15"]))

fig, ax = plt.subplots(1, 3, figsize=(18,5))
sns.heatmap(confusion_matrix(yte,pred), annot=True, fmt="d", cmap="Blues", ax=ax[0])
ax[0].set_title(f"Confusion matrix (thr={thr:.2f})"); ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual")
RocCurveDisplay.from_predictions(yte, proba, ax=ax[1]); ax[1].set_title("ROC curve")
PrecisionRecallDisplay.from_predictions(yte, proba, ax=ax[2]); ax[2].set_title("Precision-Recall curve")
plt.tight_layout(); plt.show()
```

TAB_SCORES = {"model":"1. Tabular (XGBoost)", "F1":f1, "ROC_AUC":auc, "PR_AUC":ap}

SECTION 5B — Hyperparameter tuning (Model 1)

Model 1 above used sensible hand-picked hyperparameters. Here they are tuned properly with

`RandomizedSearchCV`, so the report can state a measured improvement rather than assert one.

****Two deliberate choices.**** (1) The search is scored on `average_precision` (PR-AUC), not accuracy —

optimising accuracy on a 17%-positive target would reward the degenerate \u201calways on-time\u201d model.

(2) `RandomizedSearchCV` (30 random draws) is used over an exhaustive grid (which would be thousands

of fits) because it explores the same space at a fraction of the cost. The search is cross-validated

on the ****training split only****; the test set is untouched until the final comparison.

```
from sklearn.model_selection import RandomizedSearchCV
import time

param_grid = {
    "max_depth":       [4, 6, 8, 10],
    "learning_rate":   [0.01, 0.05, 0.1, 0.2],
    "n_estimators":    [300, 500, 800],
    "subsample":       [0.6, 0.8, 1.0],
    "colsample_bytree": [0.6, 0.8, 1.0],
    "min_child_weight": [1, 5, 10],
    "gamma":           [0, 0.1, 0.5],
}
```

```
search = RandomizedSearchCV(
   XGBClassifier(scale_pos_weight=spw, eval_metric="logloss",
                 n_jobs=-1, random_state=RANDOM_STATE),
   param_distributions=param_grid,
   n_iter=30, cv=3,
   scoring="average_precision",         # PR-AUC — imbalance-aware objective
   random_state=RANDOM_STATE, n_jobs=-1, verbose=1)

t0 = time.time()
search.fit(Xtr, ytr)                    # TRAIN split only
print(f"Search finished in {time.time()-t0:.0f}s over {len(search.cv_results_['params'])}
candidates\n")
print("Best hyperparameters:")
for k, v in search.best_params_.items():
   print(f"    {k:18s}: {v}")
print(f"\nBest CV PR-AUC: {search.best_score_:.4f}")

# ---- Tuned vs baseline on the untouched test set ----
best_clf = search.best_estimator_
p_tuned  = best_clf.predict_proba(Xte)[:, 1]

auc_t = roc_auc_score(yte, p_tuned)
ap_t  = average_precision_score(yte, p_tuned)
thr_t, f1_t = best_f1_threshold(yte, p_tuned)
_, lo_t, hi_t = bootstrap_ci(yte, p_tuned, "auc")

print("\n================ TUNED vs BASELINE (test set) ================")
print(f"{'':10s}  {'ROC-AUC':>10s}  {'PR-AUC':>8s}  {'F1':>6s}")
print(f"{'Baseline':10s}  {auc:>10.4f}  {ap:>8.4f}  {f1:>6.4f}")
print(f"{'Tuned':10s}  {auc_t:>10.4f}  {ap_t:>8.4f}  {f1_t:>6.4f}")
```

```
print(f"\nBaseline ROC-AUC 95% CI : {auc_lo:.3f} - {auc_hi:.3f}")
print(f"Tuned    ROC-AUC 95% CI : {lo_t:.3f} - {hi_t:.3f}")

if lo_t > auc_hi:
    print("\nVerdict: tuned model's CI sits ABOVE the baseline's -> a real improvement.")
elif hi_t < auc_lo:
    print("\nVerdict: tuning made it WORSE.")
else:
    print("\nVerdict: the confidence intervals OVERLAP -> tuning gave no")
    print("        statistically distinguishable gain (a legitimate finding:")
    print("        the hand-picked baseline was already near the ceiling).")

# ---- top configurations, for the report ----
cvres = pd.DataFrame(search.cv_results_)
cols  = ["mean_test_score", "std_test_score"] + [f"param_{k}" for k in param_grid]
print("\nTop 5 configurations by CV PR-AUC:")
print(cvres.sort_values("mean_test_score",
ascending=False)[cols].head(5).to_string(index=False))

# ---- search-progress plot ----
plt.figure(figsize=(11, 4))
sc = cvres.sort_values("mean_test_score").reset_index(drop=True)
plt.errorbar(range(len(sc)), sc["mean_test_score"], yerr=sc["std_test_score"],
        fmt="o", ms=4, capsize=3, color="#F5A524")
plt.axhline(ap, color="crimson", ls="--", label=f"baseline test PR-AUC = {ap:.3f}")
plt.xlabel("candidate (sorted by CV PR-AUC)"); plt.ylabel("CV PR-AUC")
plt.title("Hyperparameter search - 30 random candidates, 3-fold CV")
plt.legend(); plt.tight_layout(); plt.show()
```

```
imp = pd.Series(clf.feature_importances_,
index=PRE_DEPARTURE).sort_values(ascending=False)

plt.figure(figsize=(10,7))

sns.barplot(x=imp.values[:15], y=imp.index[:15], palette="viridis")

plt.title("Model 1 — top 15 pre-departure drivers of delay"); plt.xlabel("Importance")

plt.tight_layout(); plt.show()

print("Notice the engineered features (HOUR_HIST, ROUTE_HIST, PageRank) near the top —
")

## SECTION 6 — Model 2: Sequential (LSTM on aircraft chains)
```

Section 3.4 showed that delay ****propagates**** down an aircraft's day. Model 1 cannot use that:

it sees each flight as an isolated row. This model reads the **sequence** of legs that precede a

flight and asks whether the history of the day predicts the next leg.

```
import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models

from sklearn.preprocessing import StandardScaler


# `work` = every flight with all Section-4 features attached (train_df/test_df hold them, df
does not)

work = pd.concat([train_df, test_df]).copy()


# Per-leg features fed to the LSTM at every timestep.

# ARR_DELAY is real for previous legs and masked to 0 for the target leg (see markdown
above).

LEG_FEATS = ["DEP_HOUR_SIN", "DEP_HOUR_COS", "O_WSPD", "O_PRCP", "D_WSPD",
"D_PRCP",

        "CRS_ELAPSED_TIME", "O_PAGERANK", "D_PAGERANK", "ARR_DELAY"]

ARR_IDX = LEG_FEATS.index("ARR_DELAY")
```

```
# A proper chronological sort key: date + scheduled hour (not the raw int64 hack).
work["_DATE"]   = work["DEP_DT"].dt.date
work["_LEGKEY"] = work["DEP_DT"] + pd.to_timedelta(work["CRS_DEP_HOUR"].fillna(0),
unit="h")

# Median-fill on TRAIN medians only, then scale on TRAIN rows only.
train_ids, test_ids = set(train_df.index), set(test_df.index)
is_tr_row = work.index.isin(train_ids)
for c in LEG_FEATS:
    work[c] = work[c].fillna(work.loc[is_tr_row, c].median())

work = work.sort_values(["_DATE", "OP_CARRIER", "_LEGKEY"])
is_tr_row = work.index.isin(train_ids)                  # recompute after the sort

scaler = StandardScaler().fit(work.loc[is_tr_row, LEG_FEATS].to_numpy("float32"))
S      = scaler.transform(work[LEG_FEATS].to_numpy("float32")).astype("float32")
y_all  = work["DELAYED_15"].to_numpy("int8")
id_all = work.index.to_numpy()

# ---- Build one window per target leg ------------------------------------------------
MAXLEN  = 8
N_FEAT  = len(LEG_FEATS) + 1                            # +1 = the "is target leg" flag
seqs, labels, tgt_ids = [], [], []

pos = 0
for _, g in work.groupby(["_DATE", "OP_CARRIER"], sort=False):
    n = len(g)                                          # rows are contiguous: we sorted by these keys
    if n >= 2:
        M = S[pos:pos + n]
```

```
        for t in range(1, n):                          # t=0 has no history -> skipped
            w = M[max(0, t - MAXLEN + 1): t + 1].copy()     # window ENDS at the target leg
            w[-1, ARR_IDX] = 0.0                       # mask the target's own answer
            flag = np.zeros((len(w), 1), dtype="float32")
            flag[-1, 0] = 1.0                          # mark which timestep is the target
            seqs.append(np.hstack([w, flag]))
            labels.append(y_all[pos + t])
            tgt_ids.append(id_all[pos + t])
    pos += n

tgt_ids = np.array(tgt_ids)
y_seq   = np.array(labels)
X_seq   = tf.keras.preprocessing.sequence.pad_sequences(
            seqs, maxlen=MAXLEN, dtype="float32", padding="pre", value=0.0)

tr_idx = np.where(np.isin(tgt_ids, list(train_ids)))[0]
te_idx = np.where(np.isin(tgt_ids, list(test_ids)))[0]

print(f"Sequence samples : {len(X_seq):,}   shape = {X_seq.shape}")
print(f"  train / test   : {len(tr_idx):,} / {len(te_idx):,}")
print(f"  coverage       : {len(X_seq)/len(work):.1%} of flights "
      f"(the first leg of each carrier-day has no history to read)")
print(f"  positive rate  : {y_seq.mean():.3f}")

# ---- Train the LSTM ----------------------------------------------------------
tf.keras.utils.set_random_seed(RANDOM_STATE)

model2 = models.Sequential([
    layers.Input(shape=(MAXLEN, N_FEAT)),
```

```
    layers.Masking(mask_value=0.0),          # ignores the zero-padded timesteps
    layers.LSTM(64),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
model2.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
               loss="binary_crossentropy",
               metrics=[tf.keras.metrics.AUC(name="auc"),
                        tf.keras.metrics.AUC(name="pr_auc", curve="PR")])
model2.summary()

hist = model2.fit(
    X_seq[tr_idx], y_seq[tr_idx],
    validation_split=0.1, epochs=8, batch_size=512,
    class_weight={0: 1.0, 1: float(spw)},    # same imbalance correction as Model 1
    verbose=1)

# ---- Evaluate + export aligned probabilities for fusion ------------------------------
p2   = model2.predict(X_seq[te_idx], verbose=0).ravel()
y2   = y_seq[te_idx]

auc_2  = roc_auc_score(y2, p2)
ap_2   = average_precision_score(y2, p2)
thr_2, f1_2 = best_f1_threshold(y2, p2)
_, lo2, hi2 = bootstrap_ci(y2, p2, "auc")

print("Model 2 (Sequential / LSTM on aircraft chains):")
print(f"  ROC-AUC = {auc_2:.3f}   (95% CI {lo2:.3f}-{hi2:.3f})")
```

```
print(f"  PR-AUC  = {ap_2:.3f}    (no-skill baseline = {y2.mean():.3f})")
print(f"  F1      = {f1_2:.3f}    at tuned threshold {thr_2:.2f}")

# Per-flight probabilities indexed by flight id -> lets Section 8 fuse the three models.
p2_test_aligned = pd.Series(p2, index=tgt_ids[te_idx]).groupby(level=0).mean()
SEQ_SCORES = {"model": "2. Sequential (LSTM)", "F1": f1_2, "ROC_AUC": auc_2, "PR_AUC": ap_2}

# Training curves — evidence the model trained rather than diverged.
fig, ax = plt.subplots(1, 2, figsize=(14, 4))
ax[0].plot(hist.history["loss"], label="train"); ax[0].plot(hist.history["val_loss"], label="val")
ax[0].set_title("Loss per epoch"); ax[0].set_xlabel("epoch"); ax[0].legend()
ax[1].plot(hist.history["pr_auc"], label="train"); ax[1].plot(hist.history["val_pr_auc"], label="val")
ax[1].set_title("PR-AUC per epoch"); ax[1].set_xlabel("epoch"); ax[1].legend()
plt.tight_layout(); plt.show()

## SECTION 7 — Model 3: Graph (airport embeddings)
from scipy.sparse import coo_matrix
from sklearn.decomposition import TruncatedSVD

# ---- Adjacency from TRAIN routes only (weighted by flights flown) ----------
airports = pd.Index(sorted(set(train_df["ORIGIN"]) | set(train_df["DEST"])))
aidx     = {a: i for i, a in enumerate(airports)}
n_air    = len(airports)

r = train_df["ORIGIN"].map(aidx)
c = train_df["DEST"].map(aidx)
ok = r.notna() & c.notna()
A = coo_matrix((np.ones(int(ok.sum()), dtype="float32"),
```

```
            (r[ok].astype(int).to_numpy(), c[ok].astype(int).to_numpy())),
           shape=(n_air, n_air)).tocsr()

print(f"Adjacency: {n_air} airports x {n_air}, {A.nnz:,} directed routes, "
    f"{A.sum():,.0f} flights total")

# ---- Spectral embedding -----------------------------------------------------
EMB_DIM = 16
svd = TruncatedSVD(n_components=EMB_DIM, random_state=RANDOM_STATE)
E   = svd.fit_transform(np.log1p(A))        # log1p tames the huge hub counts
print(f"Embeddings: {E.shape}   explained variance =
{svd.explained_variance_ratio_.sum():.1%}")

def embed(frame):
    """Origin embedding || destination embedding; unseen airports -> zeros."""
    oi = frame["ORIGIN"].map(aidx).to_numpy()
    di = frame["DEST"].map(aidx).to_numpy()
    Eo = np.zeros((len(frame), EMB_DIM), dtype="float32")
    Ed = np.zeros((len(frame), EMB_DIM), dtype="float32")
    mo, md_ = ~pd.isna(oi), ~pd.isna(di)
    Eo[mo] = E[oi[mo].astype(int)]
    Ed[md_] = E[di[md_].astype(int)]
    return np.hstack([Eo, Ed])

Etr, Ete = embed(train_df), embed(test_df)
print(f"Flight-level graph features: {Etr.shape[1]} dims per flight")

# ---- Classifier on the graph embeddings -------------------------------------
extra = ["CRS_DEP_HOUR", "O_WSPD", "O_PRCP", "D_WSPD", "D_PRCP"]
Xtr3 = np.hstack([Etr, train_df[extra].fillna(0).to_numpy("float32")])
```

```
Xte3 = np.hstack([Ete,  test_df[extra].fillna(0).to_numpy("float32")])

model3 = XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05,
                       subsample=0.8, colsample_bytree=0.8, scale_pos_weight=spw,
                       eval_metric="logloss", n_jobs=-1, random_state=RANDOM_STATE)
model3.fit(Xtr3, train_df["DELAYED_15"])
p3 = model3.predict_proba(Xte3)[:, 1]

auc_3 = roc_auc_score(test_df["DELAYED_15"], p3)
ap_3  = average_precision_score(test_df["DELAYED_15"], p3)
thr_3, f1_3 = best_f1_threshold(test_df["DELAYED_15"], p3)
_, lo3, hi3 = bootstrap_ci(test_df["DELAYED_15"], p3, "auc")

print("Model 3 (Graph / SVD airport embeddings):")
print(f"  ROC-AUC = {auc_3:.3f}   (95% CI {lo3:.3f}-{hi3:.3f})")
print(f"  PR-AUC  = {ap_3:.3f}    (no-skill baseline = {test_df['DELAYED_15'].mean():.3f})")
print(f"  F1      = {f1_3:.3f}    at tuned threshold {thr_3:.2f}")

GRAPH_SCORES = {"model": "3. Graph (SVD emb)", "F1": f1_3, "ROC_AUC": auc_3,
"PR_AUC": ap_3}

# Which airports does the embedding think are alike? A quick sanity check.
from sklearn.metrics.pairwise import cosine_similarity
hub = "ATL" if "ATL" in aidx else airports[0]
sim = cosine_similarity(E[aidx[hub]].reshape(1, -1), E)[0]
near = pd.Series(sim, index=airports).drop(hub).sort_values(ascending=False).head(6)
print(f"\nAirports most similar to {hub} in embedding space:")
print(near.round(3).to_string())

## SECTION 8 — Multi-structural fusion & ablation (the payoff)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, StratifiedKFold

# ---- 1. Put every model's probability on the same index (flight id) ------------------
tab_p   = pd.Series(proba, index=test_df.index, name="tab")     # Model 1 — tabular
graph_p = pd.Series(p3,    index=test_df.index, name="graph")   # Model 3 — graph
seq_p   = p2_test_aligned.reindex(test_df.index).rename("seq")  # Model 2 — sequential

fuse = pd.concat([tab_p, graph_p, seq_p,
                  test_df["DELAYED_15"].rename("y")], axis=1)

coverage = fuse["seq"].notna().mean()
print(f"LSTM covers {coverage:.1%} of test flights "
      f"(first leg of each carrier-day has no history).")

fuse = fuse.dropna()                    # evaluate all models on the common set
yv   = fuse["y"].to_numpy()
print(f"Common evaluation set: {len(fuse):,} flights  |  positive rate {yv.mean():.3f}\n")

# ---- 2. Ablation: does each added structure actually help? ---------------------------
cv = StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE)

def stacked(cols):
    X = fuse[cols].to_numpy()
    p = cross_val_predict(LogisticRegression(max_iter=1000), X, yv,
                          cv=cv, method="predict_proba")[:, 1]
    _, lo, hi = bootstrap_ci(yv, p, "auc")
    return {"ROC_AUC": roc_auc_score(yv, p), "AUC_95%CI": f"{lo:.3f}-{hi:.3f}",
            "PR_AUC": average_precision_score(yv, p)}, p
```

```
rows, probs = [], {}
for name, cols in [
    ("Tabular only",              ["tab"]),
    ("Graph only",                ["graph"]),
    ("Sequential only",           ["seq"]),
    ("Tabular + Graph",           ["tab", "graph"]),
    ("Tabular + Sequential",      ["tab", "seq"]),
    ("Tabular + Graph + Sequential", ["tab", "graph", "seq"]),
]:
    sc, p = stacked(cols)
    sc["combination"] = name
    rows.append(sc); probs[name] = p

# naive average of all three, for contrast with the learned stacker
naive = fuse[["tab", "graph", "seq"]].mean(axis=1).to_numpy()
rows.append({"combination": "All three (naive average)",
             "ROC_AUC": roc_auc_score(yv, naive), "AUC_95%CI": "-",
             "PR_AUC": average_precision_score(yv, naive)})

ablation = (pd.DataFrame(rows).set_index("combination")
              [["ROC_AUC", "AUC_95%CI", "PR_AUC"]].round(3))
print(ablation.to_string())

# ---- 3. Is the tri-modal gain real, or inside the noise? ----------------------------
best_single = max(ablation.loc[["Tabular only", "Graph only", "Sequential only"],
"ROC_AUC"])
tri         = ablation.loc["Tabular + Graph + Sequential", "ROC_AUC"]
tri_lo, tri_hi = ablation.loc["Tabular + Graph + Sequential", "AUC_95%CI"].split("-")
verdict = ("REAL — the tri-modal CI excludes the best single structure"
```

```
        if float(tri_lo) > best_single else
        "NOT PROVEN — the intervals overlap; report this honestly")
print(f"\nBest single structure ROC-AUC : {best_single:.3f}")
print(f"Tri-modal stacker    ROC-AUC : {tri:.3f}  (95% CI {tri_lo}-{tri_hi})")
print(f"Verdict: {verdict}")

# ---- Visualise the ablation ------------------------------------------------
fig, ax = plt.subplots(1, 2, figsize=(16, 5))

colors = ["#8698B9"]*3 + ["#2FD4C0", "#2FD4C0", "#F5A524", "#C9D3E4"]
ablation["ROC_AUC"].plot(kind="barh", ax=ax[0], color=colors[:len(ablation)])
ax[0].set_xlabel("ROC-AUC (5-fold CV)"); ax[0].set_title("Does each added structure improve prediction?")
ax[0].set_xlim(0.5, max(0.80, ablation["ROC_AUC"].max() + 0.03)); ax[0].invert_yaxis()

ablation["PR_AUC"].plot(kind="barh", ax=ax[1], color=colors[:len(ablation)])
ax[1].axvline(yv.mean(), color="crimson", ls="--", label=f"no-skill = {yv.mean():.3f}")
ax[1].set_xlabel("PR-AUC (5-fold CV)"); ax[1].set_title("Same comparison on the imbalance-aware metric")
ax[1].legend(); ax[1].invert_yaxis()
plt.tight_layout(); plt.show()

# ---- Single-structure summary + ROC overlay --------------------------------
results = pd.DataFrame([TAB_SCORES, SEQ_SCORES,
GRAPH_SCORES]).set_index("model").round(3)
print("Single-structure model summary (each on its own test set):")
print(results.to_string())

fig, ax = plt.subplots(figsize=(7, 6))
for name in ["Tabular only", "Graph only", "Sequential only", "Tabular + Graph + Sequential"]:
```

```
    RocCurveDisplay.from_predictions(yv, probs[name], name=name, ax=ax)
ax.plot([0, 1], [0, 1], "k--", lw=0.8)
ax.set_title("ROC on the common test flights — three structures and their fusion")
plt.tight_layout(); plt.show()
```